TU Wien
Institute of Logic and Computation
Algorithms and Complexity Group

Heuristic Optimization Techniques, WS 2026

Programming Exercise: General Information

v1, 16-09-2026

The programming exercise consists of two assignments. This document contains information valid for both of these. Please read it carefully. If you have questions, do not hesitate to contact us either after the lectures or via `heuopt@ac.tuwien.ac.at`.

1 The Resource-Constrained k-Subtree Problem

We study the *Resource-Constrained k-Subtree Problem (RC-k-ST)*, where the goal is to find a minimum-weight subtree of a graph that spans exactly k nodes and adheres to resource constraints. Edges consume resources and have associated weights, while nodes yield resources that can later be consumed by the edges.

The problem is modeled on an undirected, not necessarily complete graph $G = (V, E)$ with $n = |V|$ nodes. We consider m resources indexed by the set $D = \{1, \dots, m\}$.

- Each node $v \in V$ yields an amount $r_v^l \geq 0$ of resource $l \in D$.
- Each edge $e = \{u, v\} \in E$ has an associated weight $w_e \geq 0$ and consumes an amount $r_e^l \geq 0$ of resource $l \in D$.

A solution is a tree $T = (V_T, E_T)$ with $V_T \subseteq V$ and $E_T \subseteq E$, i.e., a connected and acyclic subgraph of G, in which every edge of E_T has both of its endpoints in V_T. Such a tree is *feasible* if it fulfills the following two properties:

- It spans exactly k nodes, i.e., $|V_T| = k$ and consequently $|E_T| = k - 1$, where $2 \leq k \leq n$ is given as part of the instance.
- For each resource, the selected edges consume at most as much as the selected nodes yield, i.e.

$$\sum_{e \in E_T} r_e^l \;\leq\; \sum_{v \in V_T} r_v^l \qquad \forall l \in D.$$

Note that the resource constraints are imposed globally on the tree, where the budget is determined by the selected nodes and the consumption is determined by the selected edges. All instances published for this exercise admit at least one feasible solution.

The objective is to minimize the total weight of the selected edges, i.e.,

$$\min w(T) = \sum_{e \in E_T} w_e.$$

Setting all resource values to zero yields the classical k-minimum spanning tree problem [5]. As this problem is already NP-hard, so is the RC-k-ST.

A practical motivation for the RC-k-ST might be a subsidized rural broadband rollout: under a funding program, a provider commits to connecting k out of n candidate municipalities by a fiber network, and wants to minimize the long-term operation and maintenance cost of the installed links. Each municipality that is connected contributes resources to the project, e.g., its share of public subsidies, construction crew days of its local contractors, or conduit capacity it already owns, while building each fiber link consumes such resources depending on its length and the terrain. Since these resources are pooled in a joint project budget, a link does not need to be financed by its own endpoints, but the connected municipalities must jointly cover the construction of all selected links.

2 Instances & Solution Format

A problem instance is given as a plain ASCII file and contains in the first line, separated by a space, the number of nodes n, the number of edges $|E|$, the number of nodes to be spanned k, and the number of resources m. Nodes are indexed by $1, \ldots, n$. The graph is simple, i.e., it contains neither self-loops nor parallel edges, and every edge is listed exactly once.

The second part of the file, introduced by a line containing `#node resources`, contains one line per node $v = 1, \ldots, n$, holding the resources $r_v^1, \ldots, r_v^m$ yielded by that node.

The third part of the file, introduced by a line containing `#edges`, contains one line per edge $e = \{u, v\} \in E$, holding its two endpoints u and v, its weight w_e, and the resource consumptions $r_e^1, \ldots, r_e^m$ of this edge.

n $|E|$ k m
`#node resources`
r_1^1 ... r_1^m
...
r_n^1 ... r_n^m
`#edges`
u_1 v_1 w_{e_1} $r_{e_1}^1$ ... $r_{e_1}^m$
...
$u_{|E|}$ $v_{|E|}$ $w_{e_{|E|}}$ $r_{e_{|E|}}^1$ ... $r_{e_{|E|}}^m$

A solution shall be saved to a file with the following format. The first line contains the name of the instance file (without path or file extension) the solution belongs to. Each of the following $k - 1$ lines indicates one edge of E_T by its two endpoints. Neither the order of the lines nor the order of the two endpoints within a line matters. The spanned node set V_T is implied by these edges and is not listed separately.

`filename`
u_1 v_1
...
u_{k-1} v_{k-1}

All numerical values in the instances are integers, so there are no floating-point issues.

An example

Consider the instance shown in Figure 1 with $n = 6$ nodes, $|E| = 8$ edges, $k = 4$ and a single resource, i.e., $m = 1$. Its instance file and a file representing an optimal solution look as follows:

```
6 8 4 1
#node resources
3
5
2
6
1
4
#edges
1 2 4 3
1 3 6 2
2 3 2 5
2 4 5 4
3 4 3 9
4 5 7 1
4 6 1 5
5 6 8 2
```

```
example_instance
2 3
2 4
4 6
```

This solution spans the nodes $V_T = \{2, 3, 4, 6\}$, which yield $5 + 2 + 6 + 4 = 17$ units of the resource. Its edges consume $5 + 4 + 5 = 14 \leq 17$ units, so the solution is feasible, and its objective value is $w(T) = 2 + 5 + 1 = 8$.

Note that the three cheapest edges spanning these very same four nodes are $\{2, 3\}$, $\{3, 4\}$ and $\{4, 6\}$, with a total weight of only $2 + 3 + 1 = 6$. This tree consumes $5 + 9 + 5 = 19 > 17$ units and is therefore infeasible. Simply computing a minimum spanning tree on a promising set of k nodes is thus not sufficient: which edges are selected matters just as much as which nodes are selected.

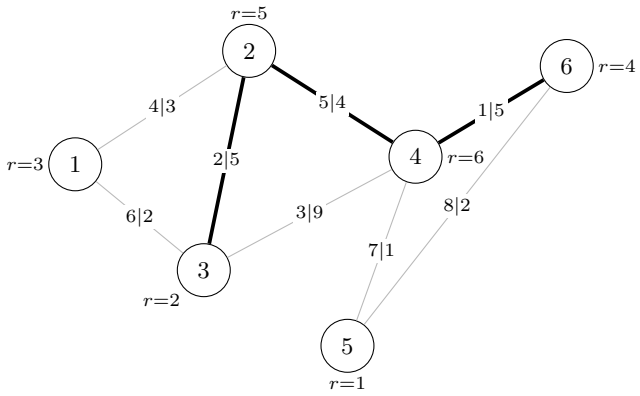


Figure 1: Example instance with $k = 4$ and $m = 1$, together with its optimal solution drawn in bold. The number inside a node is its index, the r next to it its resource yield r_v; each edge is labeled by its weight and resource consumption, $w_e|r_e$.

3 Reports

For each assignment, you are expected to hand in a concise report via TUWEL containing (unless otherwise specified) at least:

- A description of the implemented algorithms including the adaptations of the general approach, problem-specific aspects, parameters etc., not the general procedure.
- Experimental setup (machine, tested algorithm configurations).
- Best and average objective values and runtimes, plus standard deviations (for randomized algorithms over multiple runs).
- Do not use excessive runtimes for your algorithms; limit the maximum runtime to at most 15 minutes per instance on the machine you use.
- Use the instances of various sizes provided in TUWEL. Use the instances in `instances/train/` to tune the parameters of your different algorithms. Report the results on the instances in `instances/test/`.

What we do not want:

- Multithreading, multiprocessing, or GPU usage — use only a single CPU thread.
- Repetition of the problem description.

4 Solution & Source Code Submission

Hand in your best solutions for each instance and each algorithm **and** a zip-archive of your source code in TUWEL before the respective deadline (Assignment 1: Sunday, 29. 11. 2026, 23:55; Assignment 2: Sunday, 17. 01. 2027, 23:55). Make sure that the reported best solutions and the uploaded solutions match.

We will provide a competition server where you can upload solutions for the competition instances, which you find in `instances/competition/` of the instance archive in TUWEL. The uploaded solutions are then checked for correctness and, if valid, entered into a ranking table. The ranking table shows information about group rankings (best three groups per instance & algorithm) and solution values to give you an estimate of your algorithm's performance in comparison to your colleagues' algorithms. Your ranking does not influence your grade for the course. However, the three best groups at the end will win small prizes!

5 Development Environment & AC Group's Cluster

You are free to use any programming language and development environment you like.

It is also possible to use the AC group's Slurm computing cluster. Log in via ssh on `USERNAME@slurm-login01.ac.tuwien.ac.at`. The login node is only reachable from within the TU Wien network, so connect to the TU Wien VPN first when working from elsewhere. The cluster runs `Debian GNU/Linux 13` and provides you with a `gcc 14.2.0` toolchain, `Java OpenJDK 21.0.11`, `Python 3.13.5`, as well as `R 4.5.0` with `irace 4.2.0`. Further software is provided via environment modules (list them with `module avail`): `Julia 1.13.1`, `miniforge` for conda-based Python environments, and `Gurobi 13.0.3`. You may install other programming languages or language versions or software packages on your own in your home directory, but take care to stay within your disk quota (check it with `quota -s`).

A possible starting point is the following open source framework maintained by our group, which provides generic implementations of VNS, GRASP, LNS etc. and examples for the TSP, MAXSAT, and graph coloring in the Julia programming language:

- `https://github.com/ac-tuwien/MHLib.jl`

The usage of any other suitable packages, e.g., for handling graphs or visualization purposes, is also allowed.

Do not run heavy compute jobs directly on the login node but instead submit jobs to the cluster in a batch fashion.

To submit a job, create a bash script that requests the required resources, sets up your environment, and invokes your program, e.g., `job.sh`:

```
#!/bin/bash
#SBATCH --job-name=rckst
#SBATCH --time=00:15:00
#SBATCH --cpus-per-task=1
#SBATCH --mem=4G
#SBATCH --output=logs/%x-%j.out

module load julia
julia --project=. src/main.jl "$@"
```

Submit it with `sbatch job.sh [CMD_ARGS]`; additional arguments are passed on to your script. Note that Slurm does not create the `logs` directory for the output files, so create it beforehand. The script above requests a single CPU core, 4 GB of memory, and a maximum runtime of 15 minutes (hard), after which the job is killed. Please always specify `--time`, `--cpus-per-task=1`, and `--mem` so that the cluster can be shared fairly. Information about your running/pending jobs can be queried via `squeue -u $USER`. Sometimes you might want to cancel possibly wrongly submitted jobs. This can be done by `scancel <job_id>`. For many runs (e.g., all instances or multiple seeds) consider job arrays (`sbatch --array=1-30 job.sh`, with the index available as `$SLURM_ARRAY_TASK_ID`). You can find additional information under `https://ac.tuwien.ac.at/internal/help/slurm/`.

6 Further Reading

The RC-k-ST combines two well-studied lines of work. Finding a minimum-weight tree that spans exactly k nodes is known as the k-minimum spanning tree problem [5] and has been approached from a metaheuristic perspective for example in [2]. Spanning trees or subtrees under additional side constraints have likewise received considerable attention, e.g., with a fixed budget limiting a second edge attribute [4], with resources available at the nodes and consumed by each node's outgoing arcs [3], or with degree restrictions combined with a k-node requirement [1]. None of these works considers exactly the problem studied here. These references are just some possible starting points for ideas.

References

[1] Pablo Adasme and Ali Dehghan Firoozabadi. "Degree-constrained k-minimum spanning tree problem". In: *Complexity* 2020 (2020), p. 7628105.

[2] Christian Blum and Maria J. Blesa. "New metaheuristic approaches for the edge-weighted k-cardinality tree problem". In: *Computers & Operations Research* 32.6 (2005), pp. 1355–1377.

[3] Matteo Fischetti and Daniele Vigo. "A branch-and-cut algorithm for the resource-constrained minimum-weight arborescence problem". In: *Networks* 29.1 (1997), pp. 55–67.

[4] R. Ravi and Michel X. Goemans. "The constrained minimum spanning tree problem". In: *Algorithm Theory – SWAT'96*. Vol. 1097. Lecture Notes in Computer Science. Springer, 1996, pp. 66–75.

[5] R. Ravi et al. "Spanning trees – short or small". In: *SIAM Journal on Discrete Mathematics* 9.2 (1996), pp. 178–200.